Natural recursion

- The recursion happens where the data is put together
  - the natural place for it
  - recurses on sub-part of data → clear it will always terminate

```
LOX is one of:
- empty
- (cons X LOX)
```

```
(cons 1 (cons 2 (cons 3 … empty
```

```
(define (foo lox)
  (cond [(empty? lox) base]
        [else
          (fn (first lox)
              (fn-for-lox (rest lox)))]))
```

```
(fn 1
    (fn-for-lox (cons 2 (cons 3 … empty
```

Generative recursion

- The “non-natural” superset of natural recursion
- Recurses on new data → potentially does not terminate

```
(define (genrec-fn d)
  (cond [(trivial? d) (trivial-answer d)]
        [else
          (... d
               (genrec-fn (next-problem d)))]))
```

Quicksort

From Wikipedia, the free encyclopedia

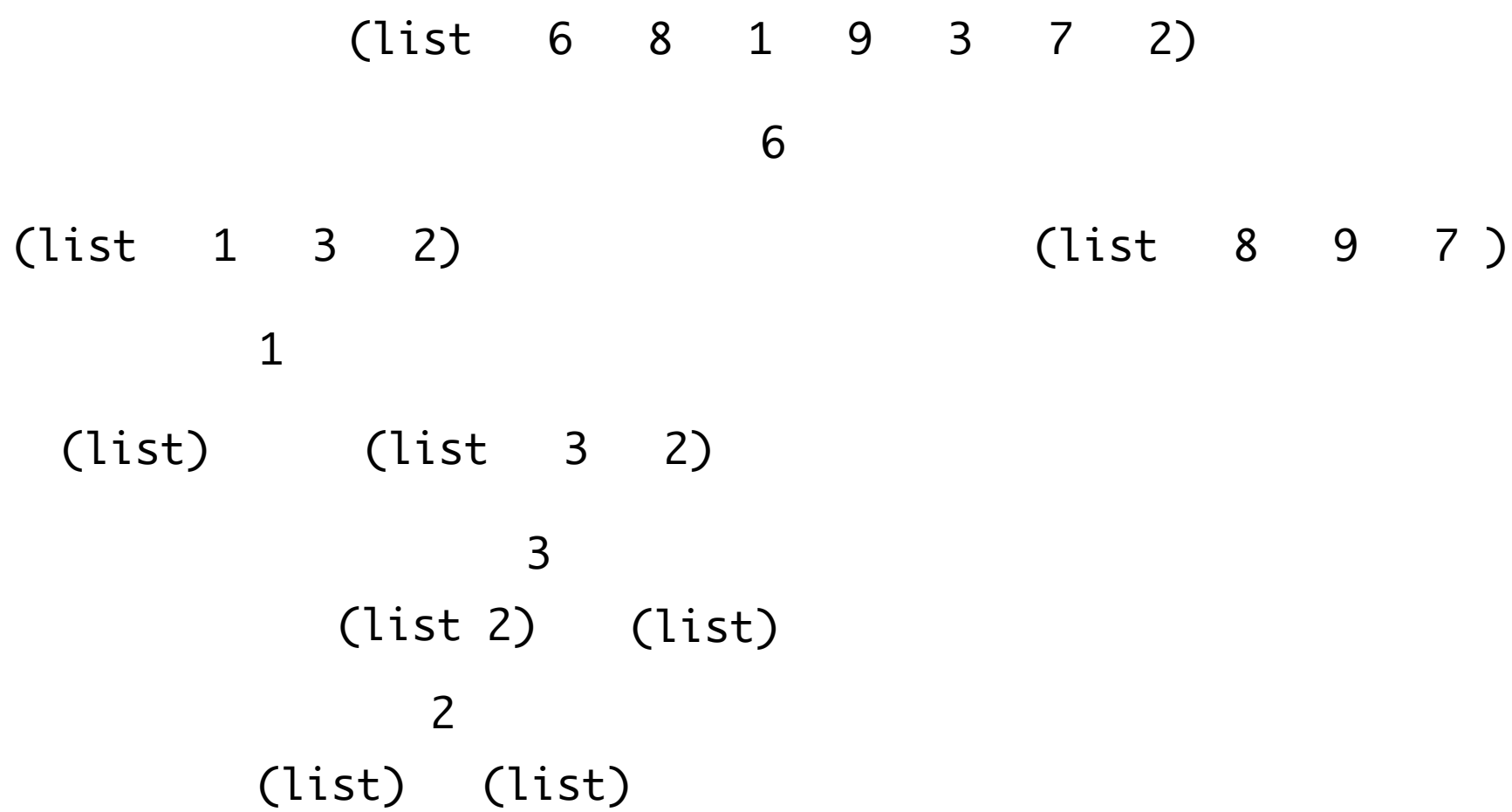
Quicksort is an ~~in-place~~ sorting algorithm. Developed by British computer scientist Tony Hoare in 1959[1] and published in 1961,[2] it is still a commonly used algorithm for sorting. When implemented well, it can be somewhat faster than merge sort and about two or three times faster than heapsort.[3][*contradictory*]

Quicksort is a divide-and-conquer algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. For this reason, it is sometimes called **partition-exchange sort**.[4] The sub-arrays are then sorted recursively. This can be done in-place, requiring small additional amounts of memory to perform the sorting.

Quicksort is a comparison sort, meaning that it can sort items of any type for which a "less-than" relation (formally, a total order) is defined. Efficient implementations of Quicksort are not a stable sort, meaning that the relative order of equal sort items is not preserved.

Mathematical analysis of quicksort shows that, on average, the algorithm takes $O(n \log n)$ comparisons to sort n items. In the worst case, it makes $O(n^2)$ comparisons.

```
(list   6   8   1   9   3   7   2)
```


(list 6 8 1 9 3 7 2)
6
(list 1 3 2)
(list 8 9 7)
1
(list)
(list 3 2)
3
(list 2)
(list)
2
(list)
(list)

Back to Design Recipes Table

Generative Recursion

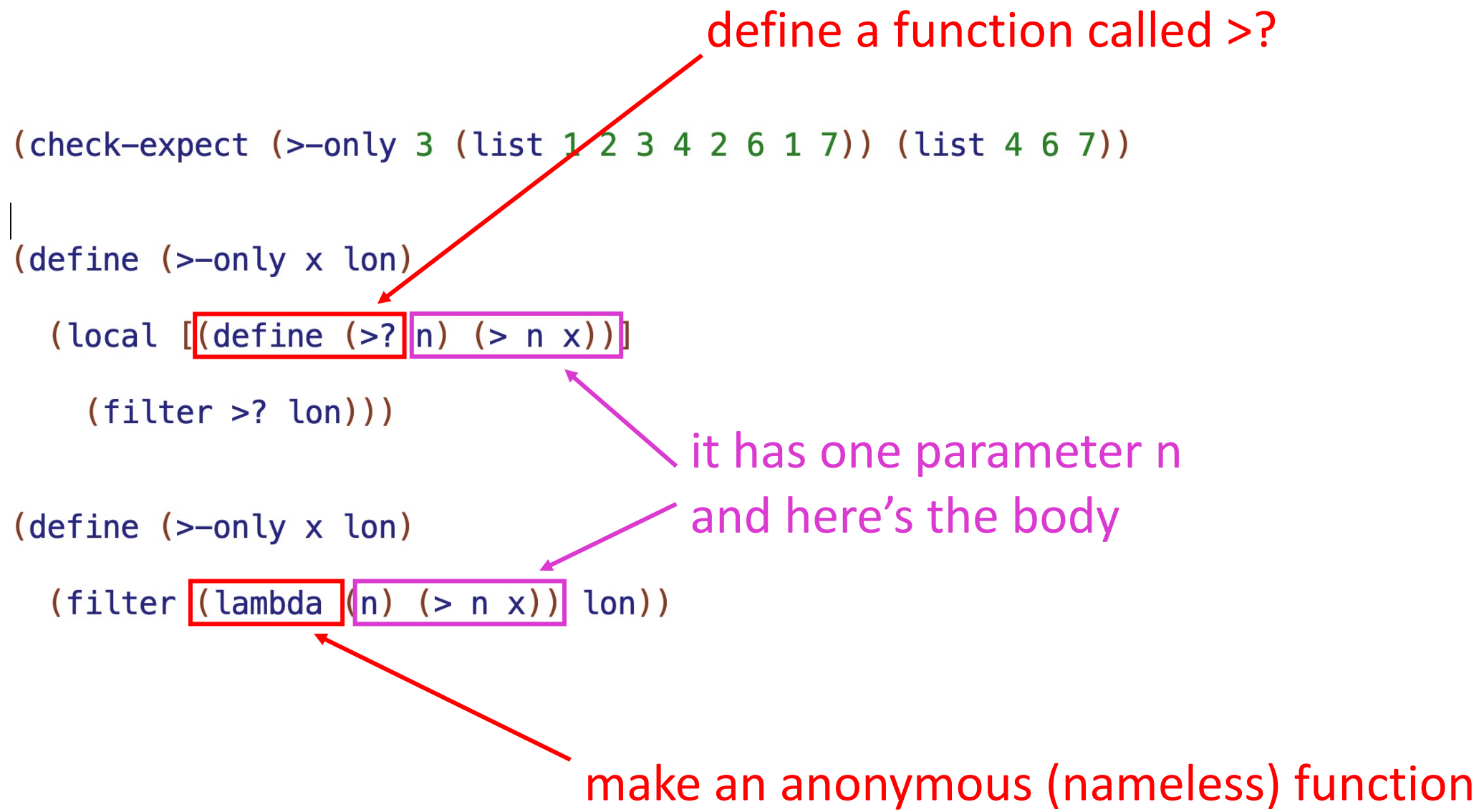
The template for generative recursion is:

```
(define (genrec-fn d)
  (cond [(trivial? d) (trivial-answer d)]
        [else
         (... d
              (genrec-fn (next-problem d)))]))
```

```
(@template-origin genrec use-abstract-fn)

(define (qsort lon)
  ;; Base case: empty
  ;; Reduction: strict subset (at least one element removed) from lon
  ;; Argument: repeated strict subset of list always reaches empty
  (cond [(empty? lon) empty]
        [else
         (local [(define p (first lon))
                 (define (<p? x) (< x p))
                 (define (>p? x) (> x p))]

           (append (qsort (filter <p? (rest lon)))
                   (list p)
                   (qsort (filter >p? (rest lon)))))]))
```

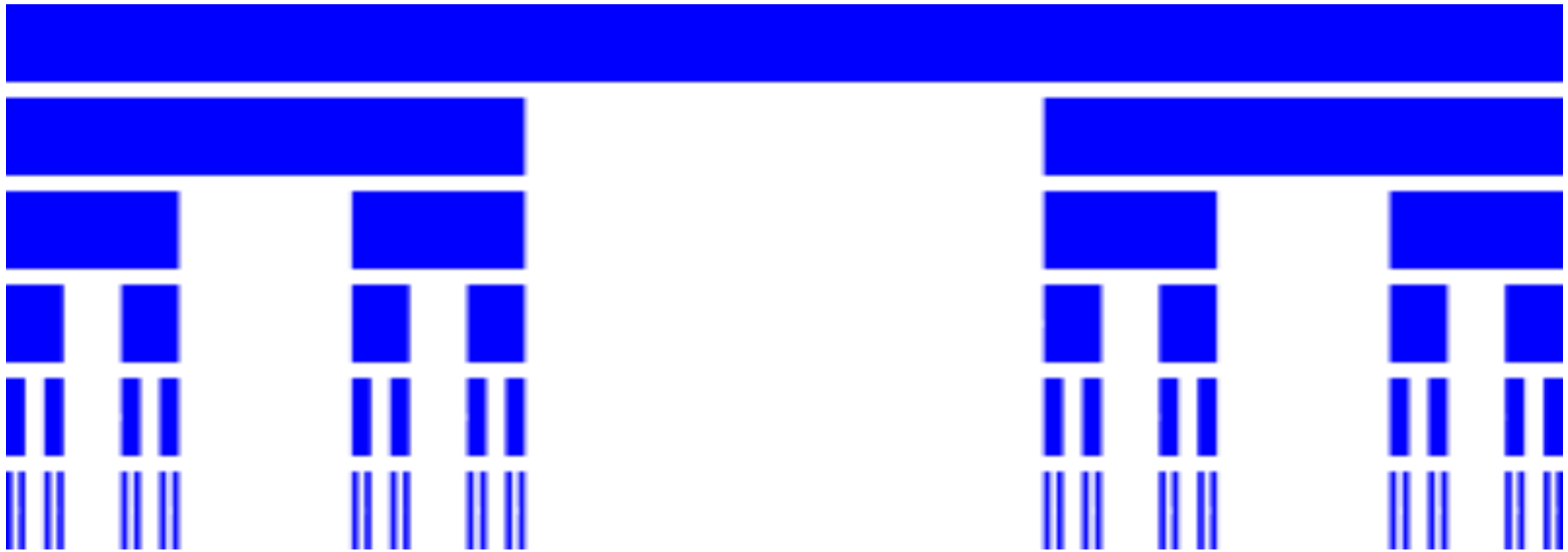

define a function called >?
(check-expect (>-only 3 (list 1 2 3 4 2 6 1 7)) (list 4 6 7))
(define (>-only x lon)
(local [(define (>? n) (> n x))]
(filter >? lon)))
(define (>-only x lon)
(filter (lambda (n) (> n x)) lon))
it has one parameter n
and here's the body
make an anonymous (nameless) function

```
(@template-origin genrec use-abstract-fn)

(define (qsort lon)
  ;; Base case: empty
  ;; Reduction: strict subset (at least one element removed) from lon
  ;; Argument: repeated strict subset of list always reaches empty
  (cond [(empty? lon) empty]
        [else
         (local [(define p (first lon))
                 (define (<p? x) (< x p))
                 (define (>p? x) (> x p))]

           (append (qsort (filter <p? (rest lon)))
                   (list p)
                   (qsort (filter >p? (rest lon))))))]))


(define (qsort lon)
  ;; Base case: empty
  ;; Reduction: strict subset of lon
  ;; Argument: repeated strict subset of list always reaches empty
  (cond [(empty? lon) empty]
        [else
         (local [(define p (first lon))]
           (append (qsort (filter (lambda (x) (< x p)) (rest lon)))
                   (list p)
                   (qsort (filter (lambda (x) (> x p)) (rest lon))))))]))
```



```
(@template-origin genrec)

(define (cantor w)
  ;; base case:  w <= CUTOFF, (note CUTOFF is > 0)
  ;; reduction:  w / 3
  ;; argument:   repeated division by 3 approaches 0, so will get below CUTOFF

  (cond [(<= w CUTOFF) (rectangle w BAR-HEIGHT "solid" BAR-COLOR)]
        [else
         (local [(define w/3 (/ w 3))
                 (define top (rectangle w BAR-HEIGHT "solid" BAR-COLOR))
                 (define gap (rectangle w GAP-HEIGHT "solid" SPACER-COLOR))
                 (define l&r (cantor w/3))
                 (define spc (rectangle w/3 BAR-HEIGHT "solid" SPACER-COLOR))]
           (above top
                  gap
                  (beside l&r spc l&r)))]))
```